

EpiGen: what every “score” in the oracle/MCMC pipeline actually means

August 15, 2026

Why this document exists

The scoring machinery grew in two places at once (this session and a parallel one working the same files), and the word “score” now refers to at least five distinct quantities, two of which are both called `wt_score` and disagree with each other. This is a precise map of what each one is, where it’s computed, what it’s used for, and where the definitions currently conflict.

Contents

1	The three underlying experts	1
2	<code>window_score</code>: the shared per-position formula	2
3	<code>combined_score</code>: what the search actually optimizes	3
4	The oracle sanity checks: <code>expert_correlation</code> and <code>fraction_below_wt</code>	3
5	The <code>wt_score</code> collision	4
6	Naming: what “WT” means at each call site	5
7	A fourth, unrelated “score”: SAE $\Delta\Delta$	5
8	Quick reference	6

1 The three underlying experts

Every score in the pipeline is built from three independent models, each queried on the *compensatory window* (`window_positions`) or on a whole candidate sequence.

ESM2 (`oracle/scoring.py`) — per-position amino-acid log-probabilities, $P(\text{aa}_i \mid \text{sequence})$, from a single unmasked forward pass. This is an *approximation* of the true masked-marginal pseudo-log-likelihood ($P(\text{aa}_i \mid \text{sequence with position } i \text{ masked})$), which would need L forward passes for a length- L sequence); the model gets to see the residue it’s predicting. Traded deliberately for speed: 1 pass instead of L .

ProteinMPNN (`oracle/scoring.py`) — per-position amino-acid log-probabilities *conditioned on the backbone structure*, $P(\text{aa}_i \mid \text{structure}, \text{context})$. Already single-pass (autoregressive teacher-forcing), no approximation needed.

Evo2 (`oracle/evo2.scoring.py`) — called here for one scalar `avg_log_likelihood` per *whole nucleotide sequence* (causal: mean of $P(x_t \mid x_{<t})$ over the sequence). Operates on DNA, not protein — the AA sequence is reverse-translated to nt via `oracle/codon.py`, and every accepted AA substitution during the search updates the corresponding codon. **Evo2 can in fact return per-position logits** (`Evo2ScoringConfig.return_logits=True` → a `(seq_len, vocab_size)` table, the same shape ESM2/ProteinMPNN already return) — `oracle/evo2.scoring.py` simply never asks for them, only ever reading `avg_log_likelihood`. See the note at the end of Section 2 for why that’s not a trivial gap to close.

Both ESM2 and ProteinMPNN expose `.batch` variants that score many sequences in a single Modal call — this is what lets the MCMC round loop recompute every chain’s proposal in one call per expert per round, instead of one call per chain.

2 window_score: the shared per-position formula

`oracle/mcmc.py`’s `window_score` is the one formula almost everything else is built from:

$$\text{window_score}(s) = \sum_{p \in \text{window_positions}} \left[w_{\text{esm2}} \cdot \text{ESM2}(p, s_p) + w_{\text{pmpnn}} \cdot \text{MPNN}(p, s_p) \right] \quad (1)$$

where s_p is the amino acid sequence s has at position p , and $\text{ESM2}(p, a) / \text{MPNN}(p, a)$ are the two experts’ precomputed per-position log-probability tables for amino acid a at position p .

Two properties matter:

- **It only sums over window_positions**, never the whole sequence. This is intentional, not a shortcut: the fixed edit and every other unchanged residue contribute an identical constant to every candidate’s score, so they cancel out of any accept/reject comparison and are simply omitted.
- **Evo2 is not in it** — not because it *can’t* be (see the correction above: it can return a per-position table too), but because the codebase doesn’t currently fetch or use that table. It is instead added as a separate whole-sequence additive term wherever the full search objective is needed (Section 3).

`window_score` is a *public* function specifically so a caller holding its own `esm2_scores/pmpnn_scores` tables (e.g. ones already spent on the oracle sanity checks, Section 4) can score an arbitrary sequence — WT, a chain’s starting point, a chain’s ending point — at zero extra Modal cost, using the exact formula the search itself optimizes minus the Evo2 term.

Could Evo2’s per-position logits be folded into window_score the same way? Not as a direct drop-in, for two reasons. First, a *vocabulary* mismatch: Evo2’s logits are over its nucleotide vocabulary, not the 20-AA **CANONICAL_AA** vocabulary `window_score` sums over — using them at an amino-acid position would mean aggregating each codon’s 3 nucleotide log-probs into one per-AA-position value first (mechanically possible via `oracle/codon.py`’s existing AA-position ↔ codon-triplet mapping, but real work, not a rename). Second, and more fundamentally, Evo2

is *autoregressive*: its logit at position t is conditioned on everything before t , unlike ESM2’s single-pass approximation or ProteinMPNN’s structure-conditioning, neither of which has that directional dependency. A table computed once on `edit_only` would be systematically wrong for any candidate that differs from `edit_only upstream` of a given window position — so even with `return_logits=True`, reusing one cached table across arbitrary candidates the way `window_score` currently reuses the ESM2/ ProteinMPNN tables would not be correct without more care (at minimum, re-fetching Evo2 logits per candidate rather than reusing one shared table).

3 combined_score: what the search actually optimizes

The number shown in the “MCMC candidates” table (`MCMCCandidate.combined_score`) and the number the round loop accepts/rejects on is:

$$\text{combined_score}(s) = \text{window_score}(s) + w_{\text{evo2}} \cdot \text{Evo2}(s) \quad (2)$$

$\text{Evo2}(s)$ is only computed (and only added) when Evo2 scoring is enabled, i.e. when a nucleotide sequence was supplied (`orchestrate.py`’s `use_evo2`, **default** `True`). When Evo2 is off, `combined_score` is `window_score` — the two coincide, which is presumably why it looked from the outside like “the score” was always just ESM2+ProteinMPNN. With Evo2 on (the default), they are not the same number.

Inconsistency: Every docstring/UI caption I wrote for the results-page histogram says “ESM2+ProteinMPNN, Evo2 excluded” — that’s accurate for what the histogram plots, but it made it look like Evo2 wasn’t part of the pipeline at all. It *is* part of `combined_score`, by default. Only the free, no-extra-Modal-call comparison plot excludes it.

4 The oracle sanity checks: `expert_correlation` and `fraction_below_wt`

Computed once, before MCMC starts, in `oracle/correlation.py`. These are diagnostics on the *full substitution space* at the window positions — every possible single-residue swap, not sequences the search actually visited.

expert_agreement Pearson correlation between ESM2’s and ProteinMPNN’s per-(position, amino acid) log-likelihood *deltas versus the reference residue’s own score* at that position. Low correlation is the desired outcome — it means the two experts are contributing independent information, not both re-deriving the same signal (per `mypipelinethoughts.md` step 4).

fraction_below_wt Fraction of (position, amino acid) substitutions whose $w_{\text{esm2}}\text{ESM2} + w_{\text{pmpnn}}\text{MPNN}$ combined score is *below* the reference residue’s own combined score at that position. A sanity check that the window isn’t uniformly hostile to substitution before trusting any candidate MCMC finds.

Inconsistency: Both functions take a parameter literally named `wt_sequence`, but `orchestrate.py` always calls them with `edit_only.sequence`, never true WT. The parameter name means “the reference sequence this comparison is relative to,” not literal wild-type — same naming looseness that caused the `mutation_name` “WT” mislabeling bug fixed earlier this session (Section 6).

5 The `wt_score` collision

This is the actual source of the confusion. **Two different quantities are both currently called `wt_score`**, computed in two different files, by two different sessions, for two different purposes:

	<code>MCMCSearchResult.wt_score</code>	<code>EndToEndResult.wt_score</code>
Where computed	<code>oracle/mcmc.py</code> , inside <code>run_mcmc_search</code> , at round 0	<code>orchestrate.py</code> , separately, after the search returns
Formula	<code>combined_score</code> of the unmutated starting chain (<code>window_score + w_evo2*Evo2</code> if Evo2 enabled)	<code>window_score</code> only, hardcoded weights 0.5/0.5, no Evo2 term ever
Used for	Per-chain <i>freeze threshold</i> : a chain stops proposing further mutations once its score exceeds this (early stopping / convergence)	The dashed vertical reference line on the results-page starting/ending-score histogram
Currently wired to UI?	No — <code>orchestrate.py</code> does not read this field at all	Yes, this is what <code>plot_score_comparison</code> actually draws

These are different numbers whenever Evo2 is enabled (the default). `orchestrate.py` currently ignores the search’s own `wt_score` / `rounds_run` / `converged_chain_count` fields entirely and recomputes a narrower version itself from scratch, using the `esm2_scores`/`pmpnn_scores` tables it already has lying around from Section 4’s sanity checks.

Why it happened

Two sessions extended `run_mcmc_search` independently on the same underlying idea (“compare against the WT/edit-only baseline”) for two different reasons: I added `starting_sequences/ending_sequences` plus a post-hoc, Modal-call-free comparison plot for the results page; the other session added early-stopping (freeze a chain once it beats the unmutated baseline) directly inside the search loop, which needed its own baseline computed *during* the search, with the full objective it’s actually optimizing, not just the window-only piece I use for the free comparison.

What this means practically right now

- The results-page histogram’s WT dashed line is **narrower** than what the search itself used to decide convergence, whenever Evo2 is on. A chain can legitimately be “frozen” (converged, per the search’s own `wt_score`) while still plotting on the wrong side of the histogram’s dashed line (per the narrower `window_score`-only baseline).
- `rounds_run` and `converged_chain_count` — real, useful information about whether the search actually finished early or ran the full **steps** — aren’t surfaced anywhere in the UI yet.
- Nothing is silently *wrong* in the sense of a crash or a raised exception; the two numbers are each internally consistent for their own purpose. They’re just both named `wt_score`, which is the actual bug.

Options going forward (not yet decided)

1. **Unify on the search’s own `wt_score`.** Drop `orchestrate.py`’s separate computation; use `mcmc_result.wt_score` (and `chain_starting_scores/ending_scores` recomputed with the Evo2 term included, at the cost of one extra Evo2 call per chain start/end) for the histogram too. Makes the plot match `combined_score` exactly; costs more Modal calls.
2. **Keep both, rename one.** E.g. rename `EndToEndResult.wt_score` to `wt_window_score` to make the scope explicit everywhere it’s displayed, and leave it intentionally Evo2-free (cheap, no extra Modal calls). Cheapest fix, doesn’t reconcile the actual numeric disagreement, just stops it being mislabeled.
3. **Surface both, explained.** Show `rounds_run / converged_chain_count` on the results page, and label the histogram’s baseline explicitly as “ESM2+ProteinMPNN only, Evo2 excluded” directly on the chart, not just in a caption.

6 Naming: what “WT” means at each call site

A recurring, related source of confusion: the string “WT” and the parameter name `wt_sequence` get reused loosely throughout the codebase to mean “the reference/baseline sequence for this particular comparison,” which is *usually* `edit_only.sequence` (WT with the fixed edit already applied), not literal wild-type.

Call site	What “reference” actually means there
<code>correlation.expert_agreement/fractionedit_only.sequence</code>	<code>edit_only.sequence</code>
<code>mcmc.run_mcmc_search’s internal <code>wt_score</code></code>	<code>folded.sequence</code> , which <code>orchestrate.py</code> always passes as <code>edit_only</code>
<code>naming.mutation_name</code> (now fixed)	Caller-supplied; <code>results.py/structure.py</code> now correctly pass true <code>wt_sequence</code> , after a bug where they passed <code>edit_only.sequence</code> and a candidate identical to it rendered as the string “WT”

The `mutation_name` case was an actual shipped bug (fixed this session): an MCMC candidate whose trajectory never found an accepted compensatory move is identical to `edit_only`, not to true WT — it still carries the fixed edit. Labeling it “WT” claimed something false. It now names against true `wt_sequence`, so such a candidate renders as just the edit itself (e.g. M1A) instead of a misleading “WT”.

7 A fourth, unrelated “score”: SAE $\Delta\Delta$

Worth naming explicitly so it doesn’t get folded into the above by accident: the SAE feature deltas (`sae_diff/run.py`, surfaced in the Structure viewer’s “Color by $\Delta\Delta$ SAE feature” dropdowns and the results page’s PCA scatter) are **not** an oracle score at all. They’re interpretability feature *activations* from ESM C’s sparse autoencoder — a completely different model, a completely different notion of “score,” with no relationship to `window_score/combined_score` beyond happening to also be a number attached to a candidate sequence. Don’t try to compare a $\Delta\Delta$ SAE value against a `combined_score` value; they’re not on the same scale or measuring the same thing.

8 Quick reference

Quantity	Includes Evo2?	Where
<code>window_score</code>	No	<code>oracle/mcmc.py</code>
<code>combined_score</code> (candi- dates table)	Yes, if enabled (default on)	<code>oracle/mcmc.py</code>
<code>MCMCSearchResult.wt_score</code> (freeze threshold)	Yes, if enabled	<code>oracle/mcmc.py</code>
<code>EndToEndResult.wt_score</code> (histogram baseline)	Never	<code>orchestrate.py</code>
<code>expert_correlation</code>	N/A (ESM2 vs MPNN agree- ment, not a fitness score)	<code>oracle/correlation.py</code>
<code>fraction_below_wt</code>	No	<code>oracle/correlation.py</code>
SAE $\Delta\Delta$	N/A (different model entirely)	<code>sae_diff/run.py</code>